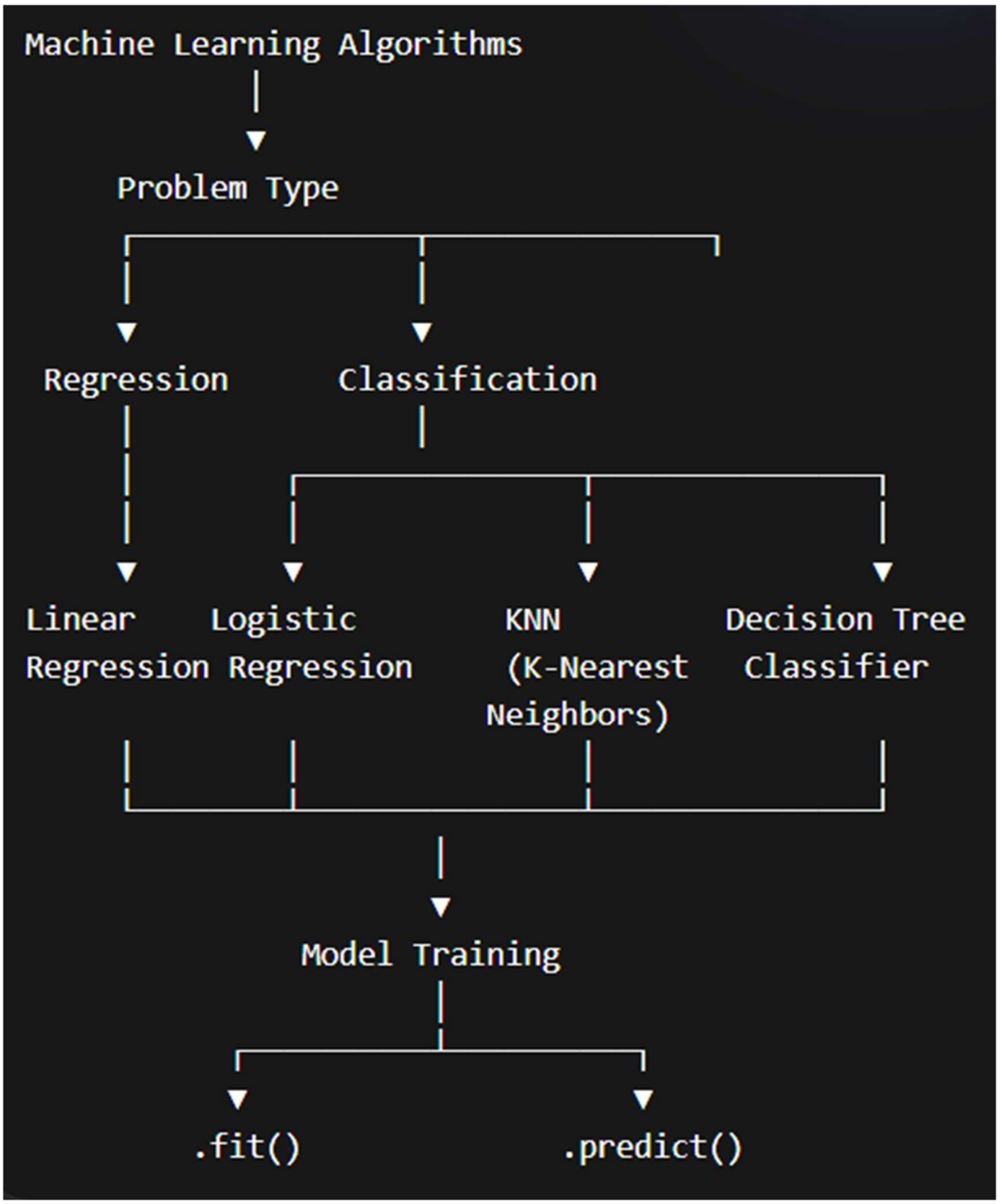
What is Regression in Machine Learning?



Regression is a type of supervised learning used to predict **continuous numerical values**.

Simple definition:
"Regression is used when the output is a number (not a category)."

Examples of Regression

- Predicting **house price**
- Predicting **salary**
- Predicting **temperature**

- Predicting **sales**

Output is always a **number (continuous value)**

Basic Idea of Regression

Regression finds a relationship between:

- **Independent Variable (X)** → Input
- **Dependent Variable (Y)** → Output

Example:

- X = Experience
- Y = Salary

Simple Linear Regression

It uses a straight line to predict values.

$$y = mx + c$$

Where:

- **y** = predicted value
- **m** = slope (relationship strength)
- **x** = input
- **c** = intercept

Types of Regression

1. Linear Regression

- Relationship between X and Y is **linear (straight line)**

Example:

- Predict salary based on experience

2. Logistic Regression

- Used for **classification**, but called regression
- Output is **probability (0 to 1)**

Example:

- Spam (1) or Not Spam (0)

3. K-Nearest Neighbors (KNN)

KNN (K-Nearest Neighbors) is a supervised learning algorithm that classifies data based on the **nearest neighbors**.

Simple:
"It checks nearby data points and decides the output."

How it works

1. Choose value of **K** (number of neighbors)
2. Find nearest K data points
3. Take **majority vote (classification)** or average (regression)

Example

- Predict if a fruit is **apple or orange** based on nearby fruits
- If most neighbors are apples → prediction = apple

Decision Tree

Decision Tree is a supervised learning algorithm that makes decisions using a **tree-like structure**.

Simple:
"It asks questions step by step and gives an answer."

How it works

- Data is split based on conditions
- Each node = a question
- Each branch = an answer
- Final node = prediction

Example

Loan Approval:

- Income > 50k?
  → Yes → Approve
  → No → Reject

Linear Regression Example (Continuous Prediction)

Problem Statement

Predict a person's **Salary** based on **Years of Experience**.

Dataset Example

Experience (Years)	Salary (₹)
1	20000

Experience (Years)	Salary (₹)
2	30000
3	40000
4	50000
5	60000

Linear Regression Idea

- Predicts **continuous values (numbers)**
- Finds a **best-fit straight line**

Formula:

$y=mx+c$

Where:

- **y** = predicted salary
- **x** = experience
- **m** = slope
- **c** = intercept

Python Code (Using sklearn)

```
# Step 1: Import libraries
import numpy as np
from sklearn.linear_model import LinearRegression

# Step 2: Create dataset
X = np.array([[1], [2], [3], [4], [5]])   # Experience
y = np.array([20000, 30000, 40000, 50000, 60000])  # Salary

# Step 3: Create model
model = LinearRegression()

# Step 4: Train model
model.fit(X, y)

# Step 5: Predict
prediction = model.predict([[6]])

print("Predicted Salary:", prediction)
```

Output Explanation

Example Output:

```
Predicted Salary: [70000]
```

Means:

- If experience = 6 years
- Predicted salary = ₹70,000

Logistic Regression Idea

Problem Statement

Predict whether a student will **Pass (1)** or **Fail (0)** based on study hours.

Dataset Example

Study Hours	Result
1	0
2	0
3	0
4	1
5	1

- Output is **probability (0 to 1)**
- Uses **sigmoid function**

Python Code (Using sklearn)

```
# Step 1: Import libraries
import numpy as np
from sklearn.linear_model import LogisticRegression

# Step 2: Create dataset
X = np.array([[1], [2], [3], [4], [5]])  # Study hours
y = np.array([0, 0, 0, 1, 1])           # Result (0 = Fail, 1 = Pass)

# Step 3: Create model
model = LogisticRegression()

# Step 4: Train model
model.fit(X, y)

# Step 5: Predict
prediction = model.predict([[3.5]])
probability = model.predict_proba([[3.5]])

print("Prediction (0=Fail, 1=Pass):", prediction)
print("Probability:", probability)
```

Output Explanation

- `predict()` → gives final class (0 or 1)
- `predict_proba()` → gives probability

Example:

```
Prediction: [1]
Probability: [[0.3 0.7]]
```

Means:

- 30% Fail

- 70% Pass

K-Nearest Neighbors (KNN) Example

Problem Statement

Predict whether a fruit is **Apple (0)** or **Orange (1)** based on its weight.

Dataset Example

Weight (grams)	Fruit
100	0 (Apple)
120	0 (Apple)
150	1 (Orange)
170	1 (Orange)

KNN Idea

- Looks at **nearest neighbors**
- Takes **majority vote**

Example:

- New fruit weight = 140
- Nearest neighbors → mostly oranges
- Prediction → **Orange**

Python Code (Using sklearn)

```
# Step 1: Import libraries
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

# Step 2: Create dataset
X = np.array([[100], [120], [150], [170]])  # Weight
y = np.array([0, 0, 1, 1])                  # 0 = Apple, 1 = Orange

# Step 3: Create model
model = KNeighborsClassifier(n_neighbors=3)

# Step 4: Train model
model.fit(X, y)

# Step 5: Predict
prediction = model.predict([[140]])

print("Prediction (0=Apple, 1=Orange):", prediction)
```

Output Explanation

Example Output:

```
Prediction: [1]
```

Means:

- 140g fruit is predicted as **Orange**

Real-Life Use Cases

- Product recommendation 🛒
- Image classification 🖼️
- Medical diagnosis 🏥

Decision Tree Classifier Example (2 Features)

Problem Statement

Predict whether a person will **Buy a Product (1)** or **Not Buy (0)** based on:

- **Age**
- **Salary**

Dataset Example

Age	Salary (₹)	Buy Product
18	20000	0
22	25000	0
25	30000	1
30	40000	1
35	50000	1

Decision Tree Idea

- Works like a **flowchart (tree structure)**
- Uses multiple conditions

Example:

- Age > 23
  → Check Salary > 28000
  → Yes → Buy (1)
  → No → Not Buy (0)

Python Code (Using sklearn)

```
# Step 1: Import libraries
import numpy as np
from sklearn.tree import DecisionTreeClassifier

# Step 2: Create dataset
# Features: [Age, Salary]
X = np.array([
    [18, 20000],
```

```
    [22, 25000],
    [25, 30000],
    [30, 40000],
    [35, 50000]
])

# Target
y = np.array([0, 0, 1, 1, 1])  # 0 = Not Buy, 1 = Buy

# Step 3: Create model
model = DecisionTreeClassifier()

# Step 4: Train model
model.fit(X, y)

# Step 5: Predict
prediction = model.predict([[28, 32000]])

print("Prediction (0=No, 1=Yes):", prediction)
```

Output Explanation

Example Output:

```
Prediction: [1]
```

Means:

- Age = 28, Salary = 32000
- Prediction = **Will Buy (1)**

Real-Life Use Cases

- Loan approval (Age + Income) 🏦
- Customer targeting 🛒
- Hiring decision 👩‍💼

Model Fitting in Machine Learning

What is Fitting?

Fitting means how well a machine learning model learns patterns from the data.

Simple:
“Fitting = How well the model understands the data.”

1. Underfitting

Underfitting happens when the model is **too simple** and cannot learn the data properly.

Simple:
“Model did not learn enough.”

Characteristics

- Poor performance on **training data**
- Poor performance on **testing data**
- High error

Example

- Trying to fit a straight line on complex (curved) data

Causes

- Model too simple
- Less training time
- Not enough features

Solution

Use a more complex model
Add more features
Train longer

2. Overfitting

Overfitting happens when the model learns **too much**, including noise.

Simple:
"Model memorizes the data instead of learning."

Characteristics

- Very good performance on **training data**
- Poor performance on **testing data**
- Low training error, high testing error

Example

- Model remembers exact data points but fails on new data

Causes

- Model too complex
- Too many features
- Small dataset

Solution

Use simpler model
Apply Regularization (Ridge, Lasso)
Use more data
Use cross-validation

3. Good Fit (Best Model)

Good Fit means the model learns patterns well and generalizes to new data.

Simple:
"Model learns correctly and works on new data."

Characteristics

- Good performance on **training data**
- Good performance on **testing data**
- Balanced error

Example

- Model captures pattern but ignores noise

Comparison Table

Feature	Underfitting	Good Fit	Overfitting
Learning	Too little	Balanced	Too much
Training Error	High	Low	Very Low
Testing Error	High	Low	High
Model Type	Too simple	Optimal	Too complex

Simple Analogy (Easy to Remember)

- **Underfitting** → Student didn't study
- **Good Fit** → Student understood concepts
- **Overfitting** → Student memorized answers